

4η Άσκηση - Ασφάλεια συνθηματικών

Γιούριϊ Οσιπιάν Π22125
Ιωάννα Ανδριανού Π22010

Χρησιμοποιώντας εργαλεία ελέγχου συνθηματικών της επιλογής σας (π.χ. δείτε τα σχετικά εργαλεία στο Kali linux και τις σχετικές εργαστηριακές σημειώσεις), για κάθε ένα από τα παρακάτω συνθηματικά, να εντοπίσετε τη hash συνάρτηση και να δοκιμάσετε να τα αποκρυπτογραφήσετε με την καταλληλότερη μέθοδο, με βάση τα δεδομένα που δίνονται σε κάθε περίπτωση. Να αναφέρετε σε κάθε περίπτωση, τη συνάρτηση hash, το συνθηματικό (εάν βρέθηκε), ποια μέθοδο χρησιμοποιήσατε (εφόσον δεν ζητείτε συγκεκριμένη μέθοδος), γιατί την επιλέξατε και πόσος χρόνος απαιτήθηκε κατά προσέγγιση. Ενδέχεται να μην είναι όλα τα συνθηματικά ευάλωτα σε επιθέσεις.

Πληροφορίες Συστήματος

Στοιχείο	Προδιαγραφές
CPU	11th Gen Intel i5-11320H (8 threads)
GPU	NVIDIA GeForce RTX 3050 Mobile
RAM	16G
OS	Linux (Arch)

Τεχνολογίες & Εργαλεία

Εργαλείο	Χρήση
Hashcat	Brute-force attacks with gpu acceleration
Hashcat	Hash type identification
hashid	Hash type identification

Προετοιμασία

Λήψη Λεξικού (English Words)

```
$ curl -LO https://github.com/dwyl/english-words/raw/master/words.zip
$ unzip words.zip
```

Στατιστικά λεξικού:

- Αρχείο: **words.txt**
- Λέξεις: ~466,551 αγγλικές λέξεις
- Μέγεθος: ~4.9 MB

1. f016441d00c16c9b912d05e9d81d894d

Επίθεση με brute force attack.

1. Αποθηκεύουμε hash

```
$ echo 'f016441d00c16c9b912d05e9d81d894d' > hash.txt
```

2. Αναγνώριση Hash Function

```
$ hashid -m hash.txt
```

```
--File 'hash.txt'--
Analyzing 'f016441d00c16c9b912d05e9d81d894d'
[+] MD2
[+] MD5 [Hashcat Mode: 0]
[+] MD4 [Hashcat Mode: 900]
[+] Double MD5 [Hashcat Mode: 2600]
[+] LM [Hashcat Mode: 3000]
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5 [Hashcat Mode: 8600]
[+] Skype [Hashcat Mode: 23]
[+] Snefru-128
[+] NTLM [Hashcat Mode: 1000]
[+] Domain Cached Credentials [Hashcat Mode: 1100]
[+] Domain Cached Credentials 2 [Hashcat Mode: 2100]
[+] DNSSEC(NSEC3) [Hashcat Mode: 8300]
[+] RAdmin v2.x [Hashcat Mode: 9900]
```

- **Επιλογή:** MD5 (πιο συνηθισμένος)

3. Επίθεση brute force attack με απλό λεξικό

```
$ hashcat -m 0 -a 0 hash.txt words.txt
```

4. Αποτελέσματα

- Συνθηματικό: f016441d00c16c9b912d05e9d81d894d:very
- Χρόνος: <1 secs

- **Παρατήρηση:** Το MD5 είναι εξαιρετικά γρήγορο αλλά **ανασφαλές** για αποθήκευση συνθηματικών. Η επίθεση ολοκληρώθηκε πολύ γρήγορα:
 1. Απλού συνθηματικού (4 χαρακτήρες)
 2. Γρήγορου αλγορίθμου MD5
 3. GPU acceleration

2. \$2b\$05\$Y3quCLZXKtKtHbHfGXQzqeqTRnbxwV/cH34idx07x.t.ukm0J7hhq

Επίθεση με brute force attack. Τι παρατηρείτε σε σχέση με το χρόνο που χρειάστηκε η επίθεση στην περίπτωση 1; Που οφείλεται αυτό;

1. Αποθηκεύουμε hash

```
$ echo '$2b$05$Y3quCLZXKtKtHbHfGXQzqeqTRnbxwV/cH34idx07x.t.ukm0J7hhq' > hash.txt
```

2. Αναγνώριση Hash Function

```
$ hashid -m hash.txt
```

```
--File 'hash.txt'--  
Analyzing '$2b$05$Y3quCLZXKtKtHbHfGXQzqeqTRnbxwV/cH34idx07x.t.ukm0J7hhq'  
[+] Unknown hash
```

```
$ hashcat --identify hash.txt
```

```
# | Name |  
Category  
=====+=====+=====  
25600 | bcrypt(md5($pass)) |  
Generic KDF  
25800 | bcrypt(sha1($pass)) |  
Generic KDF  
30600 | bcrypt(sha256($pass)) |  
Generic KDF  
28400 | bcrypt(sha512($pass)) |  
Generic KDF  
3200 | bcrypt $2*$, Blowfish (Unix) |  
Operating System  
33800 | WBB4 (Wolflab Burning Board) [bcrypt(bcrypt($pass))] |
```

Forums, CMS, E-Commerce

- **Επιλογή:** Mode 3200 (bcrypt standard)

3. Επίθεση με Hashcat

```
$ hashcat -m 3200 hash.txt words.txt
```

4. Αποτελέσματα

- Συνθηματικό: \$2b\$05\$Y3quClZXKtKtHbHfGXQzqeqTRnbxWV/cH34idx07x.t.ukm0J7hhq:**very**
 - Χρόνος: 25 secs
- **Παρατήρηση:** Συγκριτικά με το MD5 το bcrypt χρειάστηκε πολύ παραπάνω χρόνο:
 1. Key Derivation Function (KDF): Ο bcrypt σχεδιάστηκε να είναι αργός για να αποτρέπει brute-force επιθέσεις
 2. Cost Factor: Το **05** σημαίνει $2^5 = 32$ iterations (επαναλήψεις του αλγορίθμου)
 3. GPU Resistance: Ο bcrypt χρησιμοποιεί πολλή μνήμη RAM, οπότε το GPU δεν έχει μεγάλο πλεονέκτημα
 4. Σκόπιμη επιβράδυνση: Κάθε hash υπολογισμός παίρνει ~10-15ms αντί για 0.0001ms του MD5

3. 2d8ba0e99a9fc9442ded5d20ae7523df

Είναι συνδυασμός δύο αγγλικών λέξεων.

1. Αποθήκευση Hash

```
$ echo '2d8ba0e99a9fc9442ded5d20ae7523df' > hash.txt
```

2. Αναγνώριση Hash Function

```
$ hashid -m hash.txt
```

```
--File 'hash.txt'--
Analyzing '2d8ba0e99a9fc9442ded5d20ae7523df'
[+] MD2
[+] MD5 [Hashcat Mode: 0]
[+] MD4 [Hashcat Mode: 900]
[+] Double MD5 [Hashcat Mode: 2600]
[+] LM [Hashcat Mode: 3000]
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
```

```
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5 [Hashcat Mode: 8600]
[+] Skype [Hashcat Mode: 23]
[+] Snefru-128
[+] NTLM [Hashcat Mode: 1000]
[+] Domain Cached Credentials [Hashcat Mode: 1100]
[+] Domain Cached Credentials 2 [Hashcat Mode: 2100]
[+] DNSSEC(NSEC3) [Hashcat Mode: 8300]
[+] RAdmin v2.x [Hashcat Mode: 9900]
```

- **Επιλογή:** MD5 (πιο συνηθισμένος)

3. Combination Attack (2 λέξεις)

```
$ hashcat -m 0 -a 1 hash.txt words.txt words.txt
```

4. Αποτελέσματα

- Συνθηματικό: 2d8ba0e99a9fc9442ded5d20ae7523df:**tooweak**
- Χρόνος: 55 secs

- **Παρατήρηση:** Παρόλο που η πολυπλοκότητα αυξήθηκε, η επίθεση ήταν γρήγορη λόγω:

1. Ταχύτητας του MD5
2. GPU acceleration
3. Απλών λέξεων στο λεξικό

4. 2aa9bd7d6ee346b1ceedbeb6d51fbb68ba6581f10833cba0dea57a97b0445401

Είναι αγγλική λέξη με εναλλαγές πεζών-κεφαλαίων.

Φτιάχνουμε κανόνες όπου θα μετατρέπει όλες τις λέξεις σε λέξεις με εναλλαγές πεζών-κεφαλαίων (περιορισμός του rule 10 σύμβολα) Επιλέγθηκε δοκιμαστικά το πρώτο σύμβολο να είναι παιζό

```
$ cat hashcat.rule
```

```
lT0T2T4T6T8
uT1T3T5T7T9
```

1. Αποθήκευση Hash

```
$ echo '2aa9bd7d6ee346b1ceedbeb6d51fbb68ba6581f10833cba0dea57a97b0445401' > hash.txt
```

2. Αναγνώριση Hash Function

```
$ hashid -m hash.txt
```

```
--File 'hash.txt'--
Analyzing
'2aa9bd7d6ee346b1ceedbeb6d51fbb68ba6581f10833cba0dea57a97b0445401'
[+] Snefru-256
[+] SHA-256 [Hashcat Mode: 1400]
[+] RIPEMD-256
[+] Haval-256
[+] GOST R 34.11-94 [Hashcat Mode: 6900]
[+] GOST CryptoPro S-Box
[+] SHA3-256 [Hashcat Mode: 5000]
[+] Skein-256
[+] Skein-512(256)
```

- **Επιλογή:** SHA-256 (πιο συνηθισμένος)

3. Δημιουργία Rule File

```
$ vim hashcat.rule
```

```
lT0T2T4T6T8
uT1T3T5T7T9
```

4. Επίθεση με Rules

```
$ hashcat -m 1400 -a 0 hash.txt words.txt -r hashcat.rule
```

5. Αποτελέσματα

- Συνθηματικό:
2aa9bd7d6ee346b1ceedbeb6d51fbb68ba6581f10833cba0dea57a97b0445401:**CoMpLeXiTy**
- Χρόνος: <1 secs

- **Παρατήρηση:** Η επίθεση ήταν πολύ γρήγορη (<1s) λόγω:
 1. Απλό συνθηματικό
 2. Η γνώση του μοτίβου μετέτρεψε μια δύσκολη επίθεση σε απλή

5. **\$2b\$05\$0WDh2YT0ltviu7nUgflG706XM4M8B.JmfJla7eCSE40Iwjsxhkk8q** : Δεν έχουμε καμία πρόσθετη πληροφορία για αυτό το συνθηματικό.

2. Αναγνώριση Hash Function

```
$ hashid -m hash.txt
```

```
$ hashcat --identify hash.txt
```

- **Επιλογή:** Mode 3200 (bcrypt standard)

3. Επιθέσεις που δοκιμάστηκαν

```
$ hashcat -m 3200 -a 0 hash.txt words.txt
```

```
$ hashcat -m 3200 -a 0 hash.txt rockyou.txt
```

3. Αποτελέσματα

- Συνθηματικό: -
- Χρόνος: Για το μεγάλο λεξικό ~20 λεπτά

- **Παρατήρηση**

- Με χρήση hashcat και του word.txt δεν βρήκε το συνθηματικό.
- Με χρήση hashcat και του rockyou.txt από το <https://github.com/praetorian-inc/Hob0Rules/raw/master/wordlists/rockyou.txt.gz> δεν βρέθηκε συνθηματικό

4. Συμπεράσματα Ανάλυσης Αποτυχίας: Εφόσον δεν διαθέτουμε πρόσθετες πληροφορίες, οι δοκιμές πάνω στο hash θα απαιτήσουν σίγουρα μη ρεαλιστικούς χρόνους επεξεργασίας. Σε περίπτωση που το hash είναι επαρκώς ασφαλές, ο χρόνος αποκρυπτογράφησης μπορεί να εκτείνεται ακόμη και σε χρόνια